```
begin
    using Oscar: ZZ, elem_type, residue_ring, unit_group, gcd, crt, euler_phi, order,
    invmod, abelian_group, hom, preimage, gcdx
    using Latexify
end
```

Z/51

```
begin
    # Global Mathematical Domain Bindings
    Z = ZZ
    fmpz = elem_type(Z)

    # CRT parameters
    M_1 = Z(9)
    M_2 = Z(11)
    M_CRT = M_1 * M_2

    # Exponentiation parameters
    M_exp = Z(79)

    # Rings
    R_9, _ = residue_ring(Z, M_1)
    R_11, _ = residue_ring(Z, M_2)
    R_99, _ = residue_ring(Z, M_CRT)
    R_79, _ = residue_ring(Z, M_exp)

    # Abelian Groups
    A_9 = abelian_group([M_1])
    A_11 = abelian_group([M_2])
    A_99 = abelian_group([M_CRT])
    A_79 = abelian_group([M_exp])

    # Unit Groups
    U_79, mU_79 = unit_group(R_79)

    # Card Deck Size
    DECK_SIZE = Z(52)
    M_OUT = DECK_SIZE - 1 # 51
    R_51, _ = residue_ring(Z, M_OUT)
    A_51 = abelian_group([M_OUT])
end
```

# Lecture Thirteen: Enormous Exponents and Card Shuffling

## Scope

This lecture explores key applications of **modular arithmetic**, including the **Chinese Remainder Theorem (CRT)**, **perfect shuffles**, and efficient exponentiation via **successive squaring** and **seed planting**.

All implementations follow the **FP-DOD (Functional Programming & Data-Oriented Design)** methodology across **pure FP**, **abstract algebra**, and **group theory**, using **globally bound entities** from initialization.

# I. Chinese Remainder Theorem (CRT)

- Shows that every number between `1` and `m1*m2` (where `m1`, `m2` are relatively prime) has a unique **fingerprint** modulo `m1` and `m2`.
- Foundational logic for **Public key cryptography**.

## A. Statement & Solution Formula

- If $m_1$ and $m_2$ are relatively prime: $N \equiv a_1 \pmod{m_1}$ $N \equiv a_2 \pmod{m_2}$
- Since $\gcd(m_1, m_2) = 1$, by Euclid's Algorithm $\exists x, y$ such that $m_1 x + m_2 y = 1$.
- **Solution**: $N = m_1 a_2 x + m_2 a_1 y \pmod{m_1 m_2}$

# CRT Verification (2 mod 9 and 6 mod 11):

- **Pure FP**: 83
- **Abstract Algebra**: 83
- **Group Theory**: 83
- **Mathematical Proof Aggregation**: true

```
begin
    # CRT parameters
    test_a1 = Z(2)
    test_a2 = Z(6)

    # 1. Pure FP
    function solve_crt_fp(a1::fmpz, a2::fmpz)
        g, x, y = gcdx(M_1, M_2)
        N = M_1 * a2 * x + M_2 * a1 * y
        return mod(N, M_CRT)
    end

    # 2. Abstract Algebra
    function solve_crt_aa(a1::fmpz, a2::fmpz)
        return R_99(crt(a1, M_1, a2, M_2))
    end

    # 3. Group Theory
    # Same as above...
    function solve_crt_grp(a1::fmpz, a2::fmpz)
        return (crt(a1, M_1, a2, M_2) * A_99[1])[1]
    end

    ans_crt_fp = solve_crt_fp(test_a1, test_a2)
    ans_crt_aa = solve_crt_aa(test_a1, test_a2)
    ans_crt_grp = solve_crt_grp(test_a1, test_a2)

    Markdown.parse("""
    ### CRT Verification ($test_a1 mod $M_1 and $test_a2 mod $M_2):
    * **Pure FP**: **$ans_crt_fp**
    * **Abstract Algebra**: **$(ans_crt_aa.data)**
    * **Group Theory**: **$ans_crt_grp**
    * **Mathematical Proof Aggregation**: $(ans_crt_fp == ans_crt_aa.data ==
    ans_crt_grp)
    """)
end
```

# II. Perfect Shuffles

- A deck of **52 cards** is cut in half and interleaved perfectly.
- Cards numbered `0` to `51` from top to bottom.

## A. Outshuffle

- Card `0` stays on top. Outermost cards (`0`, `51`) remain on top and bottom.
- Mathematical Model (Outshuffle): $O(x) \equiv 2x \pmod{51}$ (except $O(51) = 51$)
- 8 outshuffles restore the deck.

## B. Inshuffle

- Outer cards move inside.
- Mathematical Model (Inshuffle): $I(x) \equiv 2x + 1 \pmod{53}$
- 52 inshuffles restore the deck.

## C. Discrete Magic: Send Top Card to Position `n`

- Express `n` in binary.
- Sequence based on bits (e.g. `n = 41 = (101001)`$_2$ $\rightarrow$ in-out-in-out-out-in).
- Same structural folding pattern as fast exponentiation **seed planting**!

# Outshuffle Proof (8 shuffles restores the deck):

- **Card 1 after 8 outshuffles**:
- **Pure FP**: 1
- **Abstract Algebra**: 1
- **Group Theory**: 1
- **Mathematical Proof Aggregation**: true

```
begin
    # 1. Pure FP
    function outshuffle_fp(x::fmpz, k::Int=1)
        x == Z(51) && return Z(51)
        foldl((acc, _) -> mod(acc * Z(2), M_OUT), 1:k, init=x)
    end

    # 2. Abstract Algebra
    function outshuffle_aa(x::fmpz, k::Int=1)
        x == Z(51) && return R_51(51)
        mult_factor = R_51(2)^k
        return R_51(x) * mult_factor
    end

    # 3. Group Theory
    function outshuffle_grp(x::fmpz, k::Int=1)
        x == Z(51) && return Z(51)
        return (x * Z(2)^k * A_51[1])[1]
    end

    test_card = Z(1)
    test_shuffles = 8

    ans_shuff_fp = outshuffle_fp(test_card, test_shuffles)
    ans_shuff_aa = outshuffle_aa(test_card, test_shuffles)
    ans_shuff_grp = outshuffle_grp(test_card, test_shuffles)

    Markdown.parse("""
    ### Outshuffle Proof (8 shuffles restores the deck):
    * **Card $test_card after $test_shuffles outshuffles**:
    * **Pure FP**: **$ans_shuff_fp**
    * **Abstract Algebra**: **$(ans_shuff_aa.data)**
    * **Group Theory**: **$ans_shuff_grp**
    * **Mathematical Proof Aggregation**: $(ans_shuff_fp == ans_shuff_aa.data ==
    ans_shuff_grp)
    """)
end
```

# III. Efficient Exponentiation

- Naive method (e.g., $3^{1,000,000}$) $\rightarrow$ millions of multiplications.
- Smart methods $\rightarrow$ only **dozens/hundreds**.

## A. Successive Squaring

- Compute powers of 2: $6^1, 6^2, 6^4, 6^8, \ldots$
- Decompose exponent into binary: $6^{83} = 6^{64} \times 6^{16} \times 6^2 \times 6^1$

## B. Seed Planting Method

- Count down exponents and square at each step.
- $$6 \rightarrow 6^2 \rightarrow 6^4 \rightarrow 6^4 \times 6 = 6^5 \cdots \rightarrow 6^{83}$$
- Computable modulo $m$ at each step for extreme efficiency.

# Seed Planting Exponentiation ($6^{83} \pmod{79}$):

- **Pure FP**: 34
- **Abstract Algebra**: 34
- **Group Theory**: 34
- **Mathematical Proof Aggregation**: true

```
begin
    # 1. Pure FP (Seed planting via fold over binary string)
    function seed_exp_fp(base::fmpz, exp::fmpz)
        bin_str = string(BigInt(exp), base=2)
        foldl((acc, bit) -> bit == '1' ? mod(mod(acc^2, M_exp) * base, M_exp) :
        mod(acc^2, M_exp),
            collect(bin_str)[2:end], init=mod(base, M_exp))
    end

    # 2. Abstract Algebra
    function seed_exp_aa(base::fmpz, exp::fmpz)
        bin_str = string(BigInt(exp), base=2)
        R_base = R_79(base)
        foldl((acc, bit) -> bit == '1' ? (acc^2) * R_base : acc^2,
            collect(bin_str)[2:end], init=R_base)
    end

    # 3. Group Theory
    function seed_exp_grp(base::fmpz, exp::fmpz)
        u_base = mU_79\(R_79(base))
        bin_str = string(BigInt(exp), base=2)
        # squaring = doubling the mapped abelian group generator
        res = foldl((acc, bit) -> bit == '1' ? 2 * acc + u_base : 2 * acc,
            collect(bin_str)[2:end], init=u_base)
        return mU_79(res).data
    end

    test_exp_base = Z(6)
    test_exp_pow = Z(83)

    ans_exp_fp = seed_exp_fp(test_exp_base, test_exp_pow)
    ans_exp_aa = seed_exp_aa(test_exp_base, test_exp_pow)
    ans_exp_grp = seed_exp_grp(test_exp_base, test_exp_pow)

    Markdown.parse("""
    ### Seed Planting Exponentiation \$($test_exp_base^{$test_exp_pow} \\pmod
    {$M_exp})\$:
    * **Pure FP**: **$ans_exp_fp**
    * **Abstract Algebra**: **$(ans_exp_aa.data)**
    * **Group Theory**: **$ans_exp_grp**
    * **Mathematical Proof Aggregation**: $(ans_exp_fp == ans_exp_aa.data ==
    ans_exp_grp)
    """)
end
```

```
Abelian group element [5]
```

```
(mU_79\(R_79(6)))
```